



รายงานรอบที่ 1

เรื่อง อัลกอริทึม (Automated Test Case Generation)

CMA-ES (Covariance Matrix Adaptation Evolution Strategy)

Adaptive Random Testing (ART)

โดย

นายธนินธร อันทรบุตร 673380043-6

นายศุภกร กรมรินทร์ 673380061-4

นายณัฏพล เพ็งพล 673380267-4

นายณัฐกรณ์ อินธิสาร 673380268-2

เสนอ

ผศ. ดร.ชิตสุธา สุ่มเล็ก

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา CP353201 หลักสูตรวิทยาการคอมพิวเตอร์

สาขาวิทยาการคอมพิวเตอร์ คณะวิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น ปีการศึกษา 1/2569

บทคัดย่อ

รายงานรอบที่ 1 นำเสนอผลการศึกษาเบื้องต้นของกระบวนการสร้างชุดทดสอบ (Test suite) และกรณีทดสอบ (Test case) แบบอัตโนมัติสำหรับการทดสอบระดับหน่วย (Unit testing) บนโปรแกรมภาษา Java จากชุดข้อมูล Defects4J โดยเปรียบเทียบสองแนวทางหลัก ได้แก่ (1) อัลกอริทึมเชิงค้นหาแบบ Search-based คือ CMA-ES และ Adaptive Random Testing (ART) แบบ Fixed-Size Candidate Set (FSCS-ART) และ (2) การใช้เครื่องมือ Generative AI ได้แก่ Claude (Anthropic) และ AI Sphere / IntelliSphere ของมหาวิทยาลัยขอนแก่น (ai.kku.ac.th) ผ่านการออกแบบพรอมพ์ ในรอบนี้มุ่งเน้นการทบทวนแนวคิด ขั้นตอนการทำงาน การกำหนดตัวแทนปัญหา (Problem representation) การออกแบบพรอมพ์เบื้องต้น และการวางแผนการทดลอง สำหรับรอบที่ 2 โดยยังไม่รายงานผลการวัดเชิงปริมาณ

คำสำคัญ: Automated Test Case Generation, CMA-ES, Adaptive Random Testing, Search-based Software Testing, Generative AI, Defects4J, Prompt Engineering

คำนำ

รายงานฉบับนี้จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชา CP353201 เพื่อศึกษากระบวนการสร้างกรณีทดสอบแบบอัตโนมัติ โดยคณะผู้จัดทำเลือกใช้ CMA-ES และ ART เป็นตัวแทนของแนวทาง Search-based และเลือกใช้ Claude ร่วมกับ AI Sphere / IntelliSphere เป็นตัวแทนของแนวทาง Generative AI บนชุดข้อมูล Defects4J รายงานรอบที่ 1 มีวัตถุประสงค์เพื่อสรุปความเข้าใจเชิงทฤษฎี ออกแบบแนวทางการทดลอง และเตรียมความพร้อมด้านพรอมพ์ก่อนการเก็บผลการทดลองจริงในรอบที่ 2 คณะผู้จัดทำหวังว่ารายงานฉบับนี้จะเป็นพื้นฐานที่ชัดเจนเพียงพอสำหรับการดำเนินงานในขั้นถัดไป

คณะผู้จัดทำ

ภาคการศึกษาที่ 1/2569

สารบัญ

บทคัดย่อ	ก
คำนำ	ข
สารบัญ	ค
สารบัญตาราง	ง
1. บทนำ	1
2. การศึกษากระบวนการทำงานของ Automatic Test Case Generation Algorithms	2
2.1 CMA-ES	2
2.2 ART (FSCS-ART)	3
2.3 มุมมองเชิงเปรียบเทียบ	4
2.4 สรุปเปรียบเทียบเบื้องต้น	5
3. การเลือก Generative AI / AI-Assisting Tools และการออกแบบพรอมพ์	5
4. กรอบการทดลองเบื้องต้นและคำถามวิจัย (สำหรับรอบที่ 2)	7
5. แผนการดำเนินงานต่อไป (รอบที่ 2)	8
6. สรุปผลของรายงานรอบที่ 1	8
เอกสารอ้างอิง	9

สารบัญตาราง

ตารางที่ 1	เปรียบเทียบเบื้องต้นระหว่าง CMA-ES และ ART	5
ตารางที่ 2	องค์ประกอบหลักของพรอมพ์ที่ใช้สร้าง Unit test	6

ข้อมูลสรุปของโครงการ (รอบที่ 1)

Search-based Algorithms ที่เลือก: (1) CMA-ES (Covariance Matrix Adaptation Evolution Strategy) (2) ART แบบ Fixed-Size Candidate Set (FSCS-ART)

Generative AI / AI-Assisting Tools ที่เลือก: (1) Claude (Anthropic) (2) AI Sphere / IntelliSphere
— บริการของมหาวิทยาลัยขอนแก่น (ai.kku.ac.th)

Dataset ที่ใช้ในการทดสอบ: Defects4J (<https://github.com/rjust/defects4j>)

1. บทนำ

โครงการนี้มีวัตถุประสงค์เพื่อศึกษาและเปรียบเทียบความสามารถระหว่างสองแนวทางหลักในการสร้างชุดทดสอบ (Test suite) และกรณีทดสอบ (Test case) แบบอัตโนมัติสำหรับการทดสอบระดับ Unit testing บนโปรแกรมภาษา Java ที่อยู่ในชุดข้อมูล Defects4J ได้แก่

แนวทางที่ 1 — Automatic Test Case Generation แบบ Search-based ซึ่งใช้เทคนิคการค้นหาเชิงฮิวริสติก (Metaheuristic search) เพื่อสร้างชุดทดสอบให้ครอบคลุมโค้ดและมีโอกาสตรวจพบข้อบกพร่อง โดยเลือกใช้ CMA-ES และ ART (FSCS-ART)

แนวทางที่ 2 — การใช้ Generative AI หรือ AI-Assisting Tools ในการสร้างชุดทดสอบผ่านการออกแบบพรอมป์ (Prompt engineering) โดยเลือกใช้ Claude และ AI Sphere / IntelliSphere ของมหาวิทยาลัยขอนแก่น

ในรายงานฉบับนี้ (รอบที่ 1) จะนำเสนอผลการศึกษาระบวนการทำงานของอัลกอริทึมทั้งสอง การกำหนด Representation ของปัญหา การออกแบบพรอมป์เบื้องต้นสำหรับเครื่องมือ **Generative AI** ทั้งสองตัว ตลอดจนกรอบคำถามวิจัยและแผนการทดลอง เพื่อเตรียมความพร้อมสำหรับการทดลองจริงในรอบที่ 2

2. การศึกษากระบวนการทำงานของ Automatic Test Case Generation Algorithms

2.1 CMA-ES (Covariance Matrix Adaptation Evolution Strategy)

2.1.1 แนวคิดพื้นฐาน

CMA-ES เป็นอัลกอริทึมเชิงวิวัฒนาการ (Evolutionary Algorithm) ประเภท Evolution Strategy ที่ออกแบบมาสำหรับปัญหาการหาค่าเหมาะที่สุด (Optimization) แบบต่อเนื่อง (Continuous optimization) ในปริภูมิที่มีมิติสูงและไม่ทราบรูปร่างของ Landscape ล่วงหน้า จุดเด่นของ CMA-ES คือ การปรับค่า Covariance matrix ของการแจกแจงแบบ Multivariate normal distribution ที่ใช้สุ่มประชากรใหม่ในแต่ละรุ่น (Generation) โดยอัตโนมัติ ทำให้ทิศทางการค้นหาปรับตัวเข้ากับลักษณะภูมิประเทศของฟังก์ชันเป้าหมายได้เอง โดยไม่ต้องปรับพารามิเตอร์ด้วยมือมากนัก [5], [6]

2.1.2 ขั้นตอนวิธีการทำงาน

1. เริ่มต้นด้วยจุดกึ่งกลาง (Mean vector) ค่า Step size (σ) และ Covariance matrix (C) เริ่มต้น (มักตั้งเป็น Identity matrix)
2. สุ่มสร้างประชากร (Candidate solutions) จำนวน λ ตัวจากการแจกแจง Multivariate normal distribution ที่มีค่าเฉลี่ยและ Covariance matrix ปัจจุบัน
3. ประเมินค่า Fitness ของแต่ละ Candidate solution ด้วยฟังก์ชันเป้าหมาย (ในบริบทการทดสอบซอฟต์แวร์ เช่น ค่าความครอบคลุมโค้ด หรือ Branch distance / Approximation level)
4. คัดเลือก (Selection) กลุ่มตัวที่มี Fitness ดีที่สุด (μ ตัวจาก λ ตัว) มาคำนวณจุดกึ่งกลางใหม่ (Weighted recombination)
5. ปรับค่า Covariance matrix และ Step size โดยอาศัยข้อมูลเส้นทางวิวัฒนาการ (Evolution path) เพื่อขยายหรือหดขอบเขตการค้นหาตามทิศทางที่มีแนวโน้มพบคำตอบที่ดี
6. ทำซ้ำขั้นตอนที่ 2-5 จนกว่าจะถึงเงื่อนไขหยุด (เช่น จำนวนรุ่นสูงสุด งบประมาณการประเมิน หรือค่า Fitness อยู่แล้ว)

2.1.3 การประยุกต์ใช้ในการสร้างชุดทดสอบและ Representation

ในบริบทของ Search-based Software Testing (SBST) ตัวแปรที่ CMA-ES ค้นหาคือค่าพารามิเตอร์อินพุตของ Test case โดยกำหนดฟังก์ชัน Fitness ให้สอดคล้องกับเป้าหมายความครอบคลุม เช่น Branch coverage หรือ Line coverage ผ่านค่า Branch distance และ Approximation level อัลกอริทึมจะปรับค่าพารามิเตอร์อินพุตอย่างต่อเนื่องเพื่อลดระยะห่างจากเงื่อนไขสาขา (Branch) ที่ยังไม่ถูกครอบคลุม [7], [8]

เนื่องจาก CMA-ES ทำงานบนปริภูมิต่อเนื่อง การนำไปใช้กับเมธอดภาษา Java จึงต้องกำหนด Problem representation ให้ชัดเจน ในรายงานนี้กำหนดแนวทางเบื้องต้นดังนี้ (1) พารามิเตอร์ชนิดตัวเลข (int, long, float, double) ถูกแมปเป็นค่าจริงในขอบเขตที่กำหนด แล้วปัดเศษเมื่อจำเป็น (2) ค่าบูลีนและค่าเชิงหมวดหมู่ถูกแมปผ่านช่วงต่อเนื่องแล้วแปลงกลับเป็นค่าไม่ต่อเนื่อง (3) สตริงและอ็อบเจกต์ซับซ้อนจะถูกจำกัดขอบเขตในรอบที่ 2 ด้วยการสร้างจากแม่แบบ (Template) หรือใช้ตัวสร้างค่าช่วย แล้วให้ CMA-ES ค้นหาเฉพาะพารามิเตอร์เชิงตัวเลขที่เกี่ยวข้อง แนวทางนี้ช่วยให้การทดลองเริ่มต้นได้จริงโดยไม่ขยายปริภูมิค้นหาจนเกินควบคุม

กล่าวโดยสรุป กระบวนการทำงานของ CMA-ES ในมุมมอง SQA แบ่งได้เป็น 4 ขั้นตอนหลัก ได้แก่ (1) การกำหนด Fitness Function เช่น หากต้องการเข้าใกล้เงื่อนไข $\text{if } (x - y == 0)$ อาจใช้ $|x - y|$ (2) การสุ่มคำตอบแบบกลุ่มจากการแจกแจง $\mathcal{N}(m, \sigma^2 C)$ (3) การรันและประเมินผล Fitness จากพฤติกรรมของโปรแกรม และ (4) การคัดเลือกและปรับเวกเตอร์การสุ่ม (m, C, σ) แล้ววนซ้ำจนกว่าจะครอบคลุมเป้าหมายหรือครบงบประมาณ

2.2 ART (Adaptive Random Testing) — รูปแบบ FSCS-ART

2.2.1 แนวคิดพื้นฐาน

ART เป็นเทคนิคที่พัฒนาต่อยอดจาก Random Testing (RT) โดยมีข้อสังเกตว่าข้อบกพร่อง (Fault) ในโปรแกรมมักปรากฏเป็นกลุ่มต่อเนื่องในปริภูมิอินพุต (Failure region) ดังนั้นหากเลือก Test case ที่กระจายตัวอย่างสม่ำเสมอทั่วปริภูมิอินพุต จะมีโอกาสพบ Failure region ได้เร็วขึ้นด้วยจำนวน Test case ที่น้อยกว่า RT ทั่วไป [9], [10] ในรายงานนี้เลือกใช้รูปแบบ Fixed-Size Candidate Set ART (FSCS-ART) ซึ่งเป็นรูปแบบพื้นฐานที่ถูกอ้างอิงอย่างกว้างขวาง

2.2.2 ขั้นตอนวิธีการทำงาน (FSCS-ART)

1. สุ่มสร้าง Test case แรกจากปริภูมิอินพุตแล้วเพิ่มเข้า Executed set (E)
2. สุ่มสร้าง Candidate test case จำนวนคงที่ k ตัว (Candidate set)
3. คำนวณระยะห่างระหว่างแต่ละ Candidate กับ Test case ทุกตัวใน Executed set (เช่น Euclidean distance สำหรับอินพุตตัวเลข)
4. เลือก Candidate ที่มีค่าระยะห่างขั้นต่ำ (Minimum distance) มากที่สุดเมื่อเทียบกับ Executed set ทั้งหมด มาเป็น Test case ตัวถัดไป
5. เพิ่ม Test case ที่เลือกเข้า Executed set แล้วรันโปรแกรมเพื่อตรวจสอบผลลัพธ์ (Pass/Fail) และ/หรือวัด Coverage
6. ทำซ้ำขั้นตอนที่ 2–5 จนครบงบประมาณการทดสอบ หรือจนพบ Failure

พารามิเตอร์สำคัญที่จะระบุให้ชัดในรอบที่ 2 ได้แก่ ขนาด Candidate set (k) นิยามฟังก์ชันระยะห่าง และขอบเขตของ Input domain สำหรับแต่ละเมธอดที่ทดสอบ

2.2.3 การประยุกต์ใช้ในการสร้างชุดทดสอบ

ART เหมาะสำหรับการสร้างชุดทดสอบเมื่อไม่มีข้อมูล Specification หรือ Coverage guidance ที่ซับซ้อน มาช่วยนำทางการค้นหา เนื่องจากอาศัยหลักการกระจายตัวของอินพุตเป็นหลัก ทำให้มีค่าใช้จ่ายในการคำนวณต่ำกว่าวิธี Search-based ที่ต้องประเมิน Fitness function ซับซ้อน อย่างไรก็ตาม ART มีข้อจำกัดเรื่องต้นทุนการคำนวณระยะห่างที่เพิ่มขึ้นตามจำนวน Test case ใน Executed set โดยเฉพาะเมื่อ Input space มีมิติสูง และประสิทธิภาพอาจลดลงเมื่อ Failure region มีขนาดเล็กมากหรือกระจัดกระจาย [9] ในบริบทของ Defects4J ART จะถูกใช้เพื่อสุ่มสร้างค่าพารามิเตอร์อินพุตของเมธอดเป้าหมายให้กระจายตัวครอบคลุมมากที่สุด แล้วเปรียบเทียบกับ CMA-ES และแนวทาง Generative AI ด้วยเมตริกเดียวกัน

2.3 มุมมองเชิงเปรียบเทียบ

ความแตกต่างเชิงพฤติกรรมของทั้งสองอัลกอริทึมสามารถสรุปได้ดังนี้ ART มุ่งกระจายจุดทดสอบให้ห่างกันในปริภูมิอินพุตเพื่อเพิ่มโอกาสพบ Failure region อย่างทั่วถึง โดยไม่ต้องพึ่งพาข้อมูลนำทางจาก Coverage ส่วน CMA-ES มุ่งใช้สัญญาณจาก Fitness เพื่อปรับการแจกแจงของการสุ่มให้เข้าใกล้บริเวณที่มีแนวโน้มบรรลุเป้าหมาย (เช่น สาขาที่ยังไม่ถูกครอบคลุม) กล่าวได้ว่า ART เน้นการสำรวจอย่างกว้าง

(Exploration) ในขณะที่ CMA-ES เน้นการเจาะลึกอย่างมีทิศทาง (Exploitation ที่ปรับตัวเองได้) ความแตกต่างนี้จะถูกนำไปใช้วิเคราะห์ผลการทดลองจริงในรอบที่ 2

2.4 สรุปเปรียบเทียบเบื้องต้นระหว่างสองอัลกอริทึม

ตารางที่ 1 เปรียบเทียบเบื้องต้นระหว่าง CMA-ES และ ART

มิติเปรียบเทียบ	CMA-ES	ART (FSCS-ART)
ประเภทวิธีการ	Evolutionary / Metaheuristic search	Enhanced Random Testing
กลไกหลัก	ปรับ mean, σ และ Covariance matrix ตามรุ่น	เลือก candidate ที่ห่างจาก Executed set มากที่สุด
ความต้องการ Fitness	จำเป็น (เช่น Branch distance)	ไม่จำเป็น; ใช้ระยะห่างระหว่างอินพุต
ต้นทุนการคำนวณ	สูงจากการปรับ Covariance ทุกรุ่น	เพิ่มขึ้นตามขนาด Executed set
จุดแข็งโดยสรุป	Self-adaptive เหมาะกับอินพุตตัวเลขต่อเนื่อง	เข้าใจง่าย คำนวณเบากว่าเมื่อไม่มี coverage guidance

3. การเลือก Generative AI / AI-Assisting Tools และการออกแบบพรอมพ์

3.1 เครื่องมือที่เลือกใช้

Claude (Anthropic) — เข้าถึงผ่าน Claude.ai / Claude API ใช้เป็นตัวแทนของ Generative AI เชิงพาณิชย์ระดับ Frontier model สำหรับงานสร้างโค้ดทดสอบ

AI Sphere / IntelliSphere (ai.kku.ac.th) — แพลตฟอร์ม Generative AI ที่มหาวิทยาลัยขอนแก่น ให้บริการแก่นักศึกษาและบุคลากร โดยรวมโมเดลจากหลายผู้ให้บริการไว้ในที่เดียว ในการทดลองรอบที่ 2 จะระบุชื่อและรุ่นโมเดลที่เลือกใช้บนแพลตฟอร์มนี้ให้ชัดเจน เพื่อสนับสนุนความสามารถในการทำซ้ำ (Reproducibility)

3.2 แนวทางการออกแบบพรอมพ์ (Prompt Design Strategy)

การออกแบบพรอมพ์มีเป้าหมายให้ Generative AI สร้างชุดทดสอบระดับ Unit test (JUnit) ที่ครอบคลุมโค้ดของแต่ละ Class/Method ในโปรเจกต์ Java จาก Defects4J ให้ได้มากที่สุด และมีโอกาสตรวจพบข้อบกพร่องที่ระบุไว้ในชุดข้อมูล โดยพรอมพ์จะประกอบด้วยองค์ประกอบหลักดังตารางที่ 2

ตารางที่ 2 องค์ประกอบหลักของพรอมพ์ที่ใช้สร้าง Unit test

องค์ประกอบ	รายละเอียด
1. Context / Role	กำหนดบทบาทให้ AI เป็นผู้เชี่ยวชาญด้าน Software testing ที่ทำงานร่วมกับ JUnit และ Maven/Gradle
2. Source code	แนบโค้ดของ Focal class พร้อมลายเซ็นเมธอดและ Dependency ที่จำเป็น
3. Coverage goal	ระบุเป้าหมาย เช่น Line coverage, Branch coverage และ Edge case ที่สนใจ
4. Output format	กำหนดให้ตอบเป็นโค้ด JUnit 5 ที่คอมไพล์ได้ พร้อม Assertion ที่ชัดเจน
5. Constraints	ห้ามแก้ไข Source code เดิม ห้ามใช้ไลบรารีนอก Classpath ตั้งชื่อเมธอดให้สื่อความหมาย
6. Iterative refinement	ป้อน Coverage report กลับเพื่อปรับปรุงชุดทดสอบแบบ Multi-turn

3.3 ตัวอย่างโครงร่างพรอมพ์ (Prompt Template)

โครงร่างพรอมพ์มาตรฐานที่ออกแบบไว้มีดังนี้

System/Role: คุณคือวิศวกร QA ผู้เชี่ยวชาญการเขียน Unit test ด้วย JUnit 5 สำหรับโปรเจกต์ Java ที่ใช้ Maven

Task: จงเขียนชุดทดสอบ JUnit 5 สำหรับคลาส {ClassName} ในไฟล์ {FilePath}

Source code: <แนบโค้ดของ Focal class และ Method signature ที่เกี่ยวข้อง>

Goal: ให้ชุดทดสอบครอบคลุม Branch coverage ให้ได้มากที่สุด รวมถึงกรณี Boundary value และ Exception path

Constraints: ห้ามแก้ไข Source code เดิม; ใช้เฉพาะ Library ที่มีอยู่ใน pom.xml; ตั้งชื่อเมธอดทดสอบตามรูปแบบ test<เงื่อนไข>_<ผลลัพธ์ที่คาดหวัง>

Output format: ให้ตอบเฉพาะโค้ด Java ที่คอมไพล์ได้ทันที โดยไม่มีคำอธิบายแทรกในบล็อกโค้ด

พรอมพ์ข้างต้นจะถูกปรับแต่งเป็นรายโปรเจกต์ใน Defects4J (Per-project customization) ในขั้นตอนการทดลองจริงของรอบที่ 2 และจะมีการทำ Prompt iteration เพิ่มเติมหากผลลัพธ์เบื้องต้นยังไม่ครอบคลุมเป้าหมายที่ตั้งไว้ ทั้งนี้จะบันทึกเวอร์ชันพรอมพ์และรุ่นโมเดลที่ใช้ทุกครั้งเพื่อความโปร่งใส

4. กรอบการทดลองเบื้องต้นและคำถามวิจัย (สำหรับรอบที่ 2)

4.1 คำถามวิจัย (Research Questions)

RQ1: เมื่อใช้งบประมาณการสร้างทดสอบที่เทียบเคียงกันได้ CMA-ES, ART, Claude และ AI Sphere ให้ค่าความครอบคลุมโค้ด (Line/Branch coverage) แตกต่างกันอย่างไร

RQ2: แต่ละวิธีมีความสามารถในการตรวจพบข้อบกพร่องจริงใน Defects4J (Fault detection) แตกต่างกันอย่างไร

RQ3: ข้อจำกัดเชิงปฏิบัติของแต่ละวิธี (เช่น ต้นทุนเวลา ความสามารถในการคอมไพล์ของชุดทดสอบที่ AI สร้าง และความไวต่อพารามิเตอร์) มีลักษณะอย่างไร

4.2 ขอบเขตข้อมูลและเกณฑ์คัดเลือก

เพื่อให้การทดลองทำซ้ำได้และอยู่ในขอบเขตที่เหมาะสม รอบที่ 2 จะไม่รัน Defects4J ทั้งชุดในครั้งเดียว แต่จะคัดเลือกชุดย่อย (Subset) ของโปรเจกต์/ข้อบกพร่องตามเกณฑ์เบื้องต้น เช่น มีข้อบกพร่องที่ทำซ้ำได้ มีชุดทดสอบอ้างอิง และเมธอดเป้าหมายมีพารามิเตอร์ที่แมปสู่ Representation ได้ จำนวนและความเป็นตัวแทนของชุดย่อยจะระบุชัดในรายงานรอบที่ 2

4.3 เมตริกหลัก

- 1) Line coverage และ Branch coverage ของชุดทดสอบที่สร้างได้
- 2) Fault detection rate / ความสามารถในการทำให้เกิด failing test ตามบักเป้าหมาย
- 3) ต้นทุนเชิงเวลาหรือจำนวนการประเมิน (Evaluation budget)
- 4) สำหรับแนวทาง AI: อัตราชุดทดสอบที่คอมไพล์ผ่านและจำนวนรอบ Prompt iteration ที่ใช้

5. แผนการดำเนินงานต่อไป (รอบที่ 2)

1. พัฒนาและรัน CMA-ES และ FSCS-ART กับชุดย่อยของ Defects4J ที่คัดเลือกไว้ แล้วบันทึกผล Coverage, Fault detection และต้นทุน
2. นำพรอมต์ที่ออกแบบไว้ไปใช้กับ Claude และ AI Sphere / IntelliSphere เพื่อสร้างชุดทดสอบ แล้ววัดผลด้วยเกณฑ์เดียวกัน โดยระบุรุ่นโมเดลที่ใช้
3. เปรียบเทียบผลลัพธ์ทั้ง 4 วิธี วิเคราะห์จุดแข็ง-จุดอ่อนตาม RQ1-RQ3 และสรุปผลการเรียนรู้
4. จัดเก็บ Source code, Test code, ผลการทดสอบ, Prompt, Configuration และรายการบักที่เลือกไว้บน GitHub ตามโครงสร้างที่กำหนด

6. สรุปผลของรายงานรอบที่ 1

รายงานรอบที่ 1 ได้สรุปแนวคิดและขั้นตอนของ CMA-ES กับ FSCS-ART กำหนดแนวทาง Representation เบื้องต้นสำหรับการประยุกต์ใช้กับเมธอด Java ออกแบบโครงพรอมต์สำหรับ Claude และ AI Sphere / IntelliSphere และกำหนดคำถามวิจัยพร้อมกรอบการทดลองสำหรับรอบที่ 2 งานในขั้นถัดไปคือการลงมือทดลองเชิงปริมาณบนชุดย่อยของ Defects4J และรายงานผลการเปรียบเทียบอย่างเป็นระบบ

เอกสารอ้างอิง

- [1] rjust/defects4j, GitHub repository. Available: <https://github.com/rjust/defects4j>
- [2] Defects4J Documentation. Available: https://defects4j.org/html_doc/index.html
- [3] Defects4J Dissection. Available: <https://program-repair.org/defects4j-dissection/#/>
- [4] R. Just, D. Jalali, and M. D. Ernst, “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs,” in Proc. ISSTA, 2014.
- [5] N. Hansen and A. Ostermeier, “Completely Derandomized Self-Adaptation in Evolution Strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
- [6] N. Hansen, “The CMA Evolution Strategy: A Tutorial,” arXiv:1604.00772, 2016.
- [7] P. McMinn, “Search-based Software Test Data Generation: A Survey,” *Software Testing, Verification and Reliability*, vol. 14, no. 2, pp. 105–156, 2004.
- [8] G. Fraser and A. Arcuri, “EvoSuite: Automatic Test Suite Generation for Object-Oriented Software,” in Proc. ESEC/FSE, 2011.
- [9] T. Y. Chen, H. Leung, and I. K. Mak, “Adaptive Random Testing,” in *Advances in Computer Science – ASIAN 2004*, Springer, 2004, pp. 320–329.
- [10] T. Y. Chen, F.-C. Kuo, R. G. Merkel, and T. H. Tse, “Adaptive Random Testing: The ART of Test Case Diversity,” *Journal of Systems and Software*, vol. 83, no. 1, pp. 60–66, 2010.
- [11] Anthropic, “Claude,” Available: <https://www.anthropic.com/claude>
- [12] มหาวิทยาลัยขอนแก่น, “AI Sphere / IntelliSphere,” Available: <https://ai.kku.ac.th>